

UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 5

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : SYED MAHATASIN HUSSAIN.

STUDENT ID : 23101140

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 29 - 08 - 2026

SUBMITTED TO

ATIA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Given the following processes with their **Arrival Time (AT)** and **Burst Time (BT)**:

Process	Arrival Time (AT)	Burst Time (BT)
P1	4	2
P2	2	2
P3	1	3
P4	0	6
P5	3	1

Implement the **Preemptive Shortest Job First (SJF)** CPU Scheduling Algorithm using a **Time Quantum (TQ) of 2 units**.

Calculate the following for each process:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**

Also, calculate:

- **Average Turnaround Time (Average TAT)**
- **Average Waiting Time (Average WT)**

Finally, display the results for all processes.

2. Source Code:

```
# Shortest Job First (Preemptive)

process = ["p1", "p2", "p3", "p4", "p5"]
at = [4, 2, 1, 0, 3]
bt = [2, 2, 3, 6, 1]

n = len(process)

ct = [0] * n
wt = [0] * n
tat = [0] * n

tq= 2
remaining = bt.copy()
time = 0
completed = 0

while completed < n:

    x = -1
    for i in range(n):

        if at[i] <= time and remaining[i] > 0:

            if x == -1 or remaining[i] < remaining[x]:
                x = i

    if x == -1:
        time += 1

    else:
        if remaining[x] > tq:
            time += tq
            remaining[x] -= tq
```

```
        else:
            time += remaining[x]
            remaining[x] = 0
            ct[x] = time
            completed += 1


for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]


print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i],
"\t", tat[i], "\t", wt[i])

sjf_avg_tat = sum(tat)/n
sjf_avg_wt = sum(wt)/n

print("\nAverage TAT =", sjf_avg_tat)
print("Average WT =", sjf_avg_wt)
print("\n")
```

3. Output:

```
PS G:\All Notes\Machine Learning> & "C:\Program Files\Python313\
Process AT      BT      CT      TAT     WT
p1      4       2       7       3       1
p2      2       2       4       2       0
p3      1       3       10      9       6
p4      0       6       14      14      8
p5      3       1       5       2       1

Average TAT = 6.0
Average WT = 3.2


PS G:\All Notes\Machine Learning>
```

4. Discussion:

In this experiment, **Preemptive SJF** and **Non-Preemptive SJF** CPU scheduling algorithms were implemented using the same set of five processes. The performance of both algorithms was evaluated based on **Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT, and Average WT**.

The **Preemptive SJF** algorithm produced an **Average TAT of 6.0** and an **Average WT of 3.2**. In this algorithm, a running process can be interrupted when another process with a shorter remaining execution time becomes available. This allows shorter processes to receive CPU time earlier.

On the other hand, the **Non-Preemptive SJF** algorithm produced an **Average TAT of 7.4** and an **Average WT of 4.6**. In Non-Preemptive SJF, once a process starts execution, it continues until completion and cannot be interrupted by another newly arrived process.

By comparing the results, Preemptive SJF has lower Average TAT and Average WT than Non-Preemptive SJF. Therefore, for the given set of processes, Preemptive SJF provides better overall performance.

5. Conclusion:

The experiment successfully implemented and compared **Preemptive SJF** and **Non-Preemptive SJF** scheduling algorithms. The results show that **Preemptive SJF performs better for the given processes**, with an Average TAT of **6.0** and an Average WT of **3.2**. In comparison, Non-Preemptive SJF has an Average TAT of **7.4** and an Average WT of **4.6**.

Therefore, based on the lower Average Turnaround Time and Average Waiting Time, **Preemptive SJF is the better scheduling algorithm for this particular dataset**. However, Non-Preemptive SJF is simpler to implement because once a process starts, it runs until completion.

6. GitHub:

https://github.com/Rizon40/CSE-402-Operating-System-Lab